

Comencemos con la **Cámara**. A nivel de ingeniería en Flutter, la cámara no se trata solo de "abrir la app nativa y tomar una foto" (que sería usar `image_picker`), sino de **tomar el control del flujo de hardware en tiempo real**, renderizar una vista previa (*preview*) dentro de nuestra propia interfaz y capturar el archivo.

Tutorial: Integración de la Cámara en Flutter

Este módulo práctico enseña a inicializar el hardware de la cámara, gestionar los permisos del sistema operativo, mostrar un flujo de video en tiempo real en un widget y capturar una fotografía guardándola en el almacenamiento local.

1. Fundamentos Teóricos y Conceptos Clave

Para controlar la cámara directamente, Flutter utiliza el paquete oficial `camera`.

- **CameraDescription**: Representa las características físicas de una cámara disponible en el dispositivo (si es la trasera, la frontal, su resolución máxima, etc.). El sistema operativo nos devolverá una lista con todas las cámaras detectadas.
- **CameraController**: Es el cerebro de la operación. Se encarga de inicializar el flujo de datos, encender/apagar el hardware, configurar la resolución y ejecutar la captura. **Crucial**: Este controlador consume mucha memoria RAM y energía; debe ser inicializado y, sobre todo, destruido (`dispose`) correctamente para evitar fugas de memoria (*memory leaks*).
- **CameraPreview**: Es el widget especializado que toma el flujo de datos en tiempo real del `CameraController` y lo renderiza en la pantalla de Flutter de forma eficiente.

2. Configuración del Entorno (Plataforma Nativa)

Antes de escribir código deben configurar los permisos de seguridad en los archivos nativos de Android e iOS.

En Android (`android/app/build.gradle`)

Asegúrate de que el `minSdkVersion` sea al menos **21** (requerido por el plugin de la cámara):

```
defaultConfig {  
    minSdkVersion 21  
    // ...  
}
```

En iOS (`ios/Runner/Info.plist`)

Deben añadirse las llaves que explican al usuario por qué la app requiere usar la cámara y el micrófono (necesario si se graba video):

```
<key>NSCameraUsageDescription</key>  
<string>Necesitamos acceso a la cámara para que los alumnos tomen fotos en la
```

```
práctica.</string>
<key>NSMicrophoneUsageDescription</key>
<string>Necesitamos acceso al micrófono para la grabación de video.</string>
```

3. Código Fuente

Para este enfoque práctico, estructuraremos el código en un solo archivo reproducible. Utiliza un `StatefulWidget` ya que dependemos enteramente del ciclo de vida del hardware.

Aquí tienes la estructura completa del proyecto organizada por capas (siguiendo las buenas prácticas de arquitectura limpia que se enseñan en ingeniería) y el archivo `main.dart` integrado con el módulo de la cámara.

Estructura de Carpetas Sugerida (Clean Architecture Básica)

Esta estructura es ideal porque separa la interfaz de usuario de la lógica de servicios:

```
mi_app_sensores/
├── android/
├── ios/
├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── core/
│   │   └── theme/
│   │       └── theme.dart          # Colores y estilos globales de la app
│   └── features/
│       ├── home/
│       │   └── screens/
│       │       └── home_screen.dart # Menú principal para navegar a cada sensor
│       └── camera/
│           └── screens/
│               └── camera_screen.dart # Pantalla del tutorial anterior
```

Código Fuente de los Archivos Base

Copia y pega los siguientes archivos para tener el proyecto corriendo al 100%.

1. `lib/core/theme/theme.dart`

Un estilo unificado para que la aplicación se vea profesional.

```
import 'package:flutter/material.dart';

class AppTheme {
  static ThemeData get lightTheme {
    return ThemeData(
      useMaterial3: true,
      colorScheme: ColorScheme.fromSeed(
        seedColor: Colors.blueAccent,
        primary: Colors.blueAccent,
        secondary: Colors.teal,
      ),
      appBarTheme: const AppBarTheme(
        backgroundColor: Colors.blueAccent,
        foregroundColor: Colors.white,
        centerTitle: true,
        elevation: 2,
      ),
    );
  }
}
```

2. lib/features/home/screens/home_screen.dart

La central de control. Aquí se irán desbloqueando los botones conforme completes los siguientes tutoriales (GPS, Bluetooth, etc.).

```
import 'package:flutter/material.dart';
import '../camera/screens/camera_screen.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Laboratorio de Sensores'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: GridView.count(
          crossAxisCount: 2, // 2 columnas
          crossAxisSpacing: 16,
          mainAxisSpacing: 16,
          children: [
            // Tarjeta para el Sensor 1: Cámara
            _SensorCard(
              title: 'Cámara',
              icon: Icons.camera_alt,
```

```

        color: Colors.blue,
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (context) => const CameraScreen()),
          );
        },
      ),

      // Tarjetas inactivas que se activarán en las siguientes clases:
      _SensorCard(
        title: 'GPS (Próximamente)',
        icon: Icons.location_on,
        color: Colors.grey,
        onTap: () => _showComingSoon(context, 'GPS'),
      ),
      _SensorCard(
        title: 'Wi-Fi (Próximamente)',
        icon: Icons.wifi,
        color: Colors.grey,
        onTap: () => _showComingSoon(context, 'Wi-Fi'),
      ),
      _SensorCard(
        title: 'Bluetooth (Próximamente)',
        icon: Icons.bluetooth,
        color: Colors.grey,
        onTap: () => _showComingSoon(context, 'Bluetooth'),
      ),
      _SensorCard(
        title: 'Micrófono (Próximamente)',
        icon: Icons.mic,
        color: Colors.grey,
        onTap: () => _showComingSoon(context, 'Micrófono'),
      ),
    ],
  ),
),
);
}

void _showComingSoon(BuildContext context, String sensor) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('La práctica de $sensor se habilitará en la siguiente sesión.')),
  );
}

class _SensorCard extends StatelessWidget {
  final String title;
  final IconData icon;
  final Color color;
  final VoidCallback onTap;

```

```

const _SensorCard({
  required this.title,
  required this.icon,
  required this.color,
  required this.onTap,
});

@override
Widget build(BuildContext context) {
  return InkWell(
    onTap: onTap,
    borderRadius: BorderRadius.circular(16),
    child: Ink(
      decoration: BoxDecoration(
        color: color.withOpacity(0.15),
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: color, width: 2),
      ),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(icon, size: 48, color: color),
          const SizedBox(height: 12),
          Text(
            title,
            textAlign: TextAlign.center,
            style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color:
color.withOpacity(0.9)),
          ),
        ],
      ),
    ),
  );
}

```

3. lib/features/camera/screens/camera_screen.dart

(Mantenemos la lógica explicada anteriormente, lista para producción)

```

import 'package:flutter/material.dart';
import 'package:camera/camera.dart';
import 'dart:io';

class CameraScreen extends StatefulWidget {
  const CameraScreen({super.key});

  @override
  State<CameraScreen> createState() => _CameraScreenState();
}

```

```
class _CameraScreenState extends State<CameraScreen> {
  List<CameraDescription> _cameras = [];
  CameraController? _controller;
  bool _isInitialized = false;
  XFile? _capturedImage;

  @override
  void initState() {
    super.initState();
    _setupCameras();
  }

  Future<void> _setupCameras() async {
    try {
      _cameras = await availableCameras();
      if (_cameras.isNotEmpty) {
        _controller = CameraController(
          _cameras[0],
          ResolutionPreset.medium,
          enableAudio: false,
        );
        await _controller!.initialize();
        if (!mounted) return;
        setState(() {
          _isInitialized = true;
        });
      }
    } catch (e) {
      debugPrint("Error inicializando cámara: $e");
    }
  }

  Future<void> _takePicture() async {
    if (_controller == null || !_controller!.value.isInitialized ||
    _controller!.value.isTakingPicture) {
      return;
    }
    try {
      XFile file = await _controller!.takePicture();
      setState(() {
        _capturedImage = file;
      });
    } catch (e) {
      debugPrint("Error al tomar foto: $e");
    }
  }

  @override
  void dispose() {
    _controller?.dispose();
    super.dispose();
  }
}
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Sensor: Cámara')),
    body: _isInitialized
      ? Column(
          children: [
            Expanded(
              flex: 2,
              child: Padding(
                padding: const EdgeInsets.all(12.0),
                child: ClipRRect(
                  borderRadius: BorderRadius.circular(16),
                  child: CameraPreview(_controller!),
                ),
              ),
            ),
            Expanded(
              flex: 1,
              child: Row(
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                children: [
                  Container(
                    width: 100,
                    height: 130,
                    decoration: BoxDecoration(
                      border: Border.all(color: Colors.grey),
                      borderRadius: BorderRadius.circular(12),
                    ),
                    child: _capturedImage != null
                      ? ClipRRect(
                          borderRadius: BorderRadius.circular(10),
                          child: Image.file(File(_capturedImage!.path), fit:
BoxFit.cover),
                        )
                      : const Center(child: Text('Sin foto', style:
TextStyle(fontSize: 12))),
                    ),
                  FloatingActionButton.large(
                    onPressed: _takePicture,
                    child: const Icon(Icons.camera_alt),
                  ),
                ],
              ),
            ),
          ],
        )
      : const Center(child: CircularProgressIndicator()),
  );
}

```

4. lib/main.dart

El punto de entrada de la aplicación. Configura el tema global y levanta la pantalla inicial (`HomeScreen`).

```
import 'package:flutter/material.dart';
import 'core/theme/theme.dart';
import 'features/home/screens/home_screen.dart';

void main() {
  // Asegura que los bindings nativos de los sensores estén listos antes de
  // arrancar la app
  WidgetsFlutterBinding.ensureInitialized();

  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Ingeniería de Sensores Móviles',
      theme: AppTheme.lightTheme,
      home: const HomeScreen(),
    );
  }
}
```